Interview Coach

Apertus-based interview practice for teenagers

Next Question — Alisa Smirnova | Track 2A, FHGR

alisa.mihalna@gmail.com

15 October 2026

GitHub repository | *Video URL to be added before submission*

1. Summary

Young people preparing for a first apprenticeship or job interview need a place to practise without the pressure of a real selection process. Interview Coach runs a text conversation, presents interview questions, offers a final recap and lets the user choose an optional exercise and revise an answer. German, French and Italian are the evaluated languages; English is also available for debugging and practice.

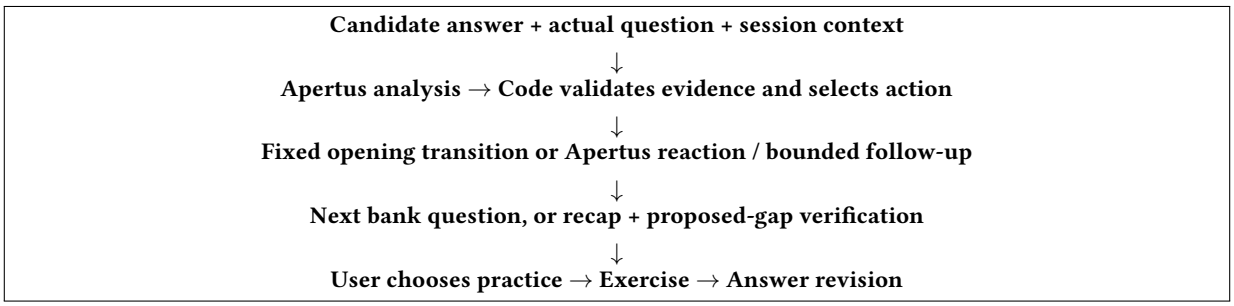
The design combines a code-controlled interview plan with Apertus for language understanding, conversational responses and feedback. Separating progression from generation keeps question coverage, follow-up bounds and termination under application control. A second model task checks proposed improvement gaps against the candidate's complete answers. This is an engineering combination of established methods, rather than a claim of a new learning algorithm.

Development evaluation covered 28 interview scenarios containing 416 candidate answers. The recorded conversation-and-recap workload averaged 2.83 coach calls per answer, with a maximum session average of 3.50. These figures show that the approach can operate within a small request budget, but they do not establish coaching quality. The application runs through Docker; conversational grounding and local model-serving requirements remain areas for further work.

This project is intended as a prototype and an exploration of what can be achieved with Apertus inference and lightweight code control, without complex agent frameworks or model training, within limited development time. It is also my first experience building an LLM-based application. After a long career break, my main motivation was to learn by building something practical, gain a new relevant skill and explore how this technology could support young people.

2. Architecture

The application has an HTML/CSS/JavaScript interface and a dependency-free Python server. Python controls interview progression and request accounting; Apertus runs at a configured remote endpoint. Docker packages the application and public resources, while model weights remain outside the container. The interface supports language and occupation selection, early recap and optional answer revision, with local persistence and basic fault-tolerance for interrupted requests.



The controller tracks stage, question subtype, covered topics, main-question and follow-up counts, actual spoken wording, candidate evidence and the transcript. The browser plan contains thirteen main questions. Eligible questions allow at most one clarification; small talk, candidate questions and closing do not. Users may end early with a recap. Coverage records what was discussed, not demonstrated competence; mastery-based selection is deferred.

Feedback, exercises and revisions form one conversation. Practice is voluntary and does not replace the original interview evidence used for assessment.

3. Use of Apertus

Development inference used `swiss-ai/Apertus-v1.5-8B`. All live model roles use the same configured Apertus endpoint; they are different tasks and prompts, not different model weights. There is no fine-tuning, adapter training or live external judge.

Task	Inputs and responsibility
Analysis	Actual question and separate bank wording, stage/subtype, latest answer, dialogue and prior evidence. Extract quoted observations, completeness, a specific missing detail, concerns and company-information requests.
Speaker	Code-selected action, validated analysis, actual answer, fictional posting and next question. Write a short reaction, company reply, supportive acknowledgement or one permitted clarification. Code appends advancing bank questions.
Recap	Public rubric and guidelines, complete question–answer pairs and criterion-eligible evidence IDs. Produce assessments and separate, specific improvement proposals.
Verification	Proposed gaps, complete answers and eligible evidence. Check whether an alleged gap is actually missing, relevant or contradicted by a later answer.
Learning	Only after explicit user choice: selected topic, actual question, original answer and evidence, excluding rejected proposals and unverified assessment text. Generate an exercise and reflection.
Revision	Assigned exercise, original answer, most recent accepted answer and new answer. Give short feedback on visible changes; save the new answer for the next comparison.

Ordinary, adequately answered greetings and self-introductions in German, French and Italian use fixed translated transitions after analysis, avoiding a speaker call. Explicit concerns, company questions and incomplete introductions retain model handling. English remains model-driven. The initial opening requires no model call. Substantive answers normally require analysis and speaking. Recap criteria are grouped according to the remaining request budget; a separate verification request runs when proposals exist. No exercise is assigned automatically. Unsupported negative or dependent assessments are withheld, while explicitly independent positive assessments may retain their quotations. Exact quotations do not establish correct interpretation.

JSON schemas and code validate fields, quote presence, evidence IDs and question bounds. Each task allows at most one correction attempt. Attempts and retries consume a session-wide budget targeting fewer than five coach requests per accepted interview/practice answer.

4. Data

Runtime resources come from the organiser's FHGR Hackapertus datasets: the interview question bank, fictional apprenticeship postings, the eleven-criterion candidate rubric and constructive-feedback guidelines. Copies and source SHA-256 hashes are stored under `track_2a/data/interview/`. The candidate rubric assesses answers; the separate feedback guidelines assess coach behaviour.

The evaluation cohort comprises 20 German, five French and three Italian scenarios. A separate Apertus simulator generated candidate answers from fictional profiles in response to actual interviewer questions. Personas and scenario success criteria were withheld from the coach. Reference annotations are not live coach inputs. No recruited teenagers or human learning outcomes are included in this evaluation.

Generated conversations, reference annotations and credentials remain local and Git-ignored. Real practice answers are sent to the configured provider and saved on the local server; users are asked to avoid identifying details. Organiser materials retain FHGR/Profolio attribution and their challenge-use conditions; occupation information originates from berufsberatung.ch/SDBB. They are not assumed to be freely relicensable under the application licence.

5. Evaluation

We measured interview completion, model-request efficiency and feedback quality across all 28 organiser scenarios: 20 German, five French and three Italian, with 416 candidate answers.

Python scripts calculated completion and request counts, including correction attempts. GPT-6.1-Sol assisted the qualitative assessment of complete transcripts and final feedback against the organiser's constructive-feedback guidelines: relevance, supporting evidence, unsupported praise or claims, understandable language, conversational continuity and useful next steps. This assesses coach behaviour, separately from Apertus's in-app scoring of candidate answers.

Instructions were issued interactively, without a single saved original judge prompt. A reconstruction of the evaluation protocol can be found in `evaluation_protocol.txt` in the repository. No calibrated overall quality score or repeated-run consistency score was established.

Measure	Result
Interview scenarios / candidate answers	28 / 416
Recorded conversation-and-recap requests	1,178
Average requests per answer	2.83
Highest session average	3.50
Recaps returned / sessions offering voluntary practice	28 / 28

What the results tell us. All supplied situations were covered within a modest request budget; the highest recorded session average was below five calls per answer. Every evaluated session offered voluntary practice rather than assigning an exercise automatically. This establishes a functioning workflow and user control, not that every response was useful.

The qualitative assessment found generally courteous language and relevant occupation-specific questions. However, reactions sometimes repeated a template or assumed unmentioned experiences, and assessments could overstate their evidence. Some recap assessments were omitted after incomplete or invalid model responses, especially in French and Italian; receiving a recap does not mean every criterion was assessed.

Overall, request efficiency is promising, while conversational and feedback quality remain inconsistent.

6. Limitations

Completing an interview is different from conducting it well. The application can control progression and validate output structure while still missing an answer's meaning. The main limitations are:

- **Grounding and attention.** Reactions can repeat examples, invent an unmentioned experience or praise a quality not established by the answer. Follow-ups can move to an unrelated topic. Fixed opening transitions address a narrow case, not this broader problem.
- **Feedback reliability.** Literal quotes can support the wrong conclusion. The same-model verifier can approve irrelevant gaps or reject useful ones. Conservative withholding leaves some criteria unassessed; truncation and validation failures are especially visible in French and Italian.
- **Adaptation and learning.** No clarification was exercised in the retained full-interview cohort. Voluntary exercises can be generic, and support for strong self-doubt remains insufficient. Learning benefit, dialect comprehension and repeated-run consistency are unmeasured.
- **Deployment gate.** The Docker image is an API client that uses a remote Apertus endpoint and not a locally served model.

7. Reproducibility

The application snapshot used to prepare this report is Git commit `4343e07`. Evaluation metrics and supporting traces remain local with source hashes. The published repository provides the application, public runtime resources, prompts, tests and deployment instructions.

Install Docker Desktop or a compatible engine. Create ignored `track_2a/data/.env` with the provider's actual values:

```
LLM_API_KEY=your-key
LLM_BASE_URL=https://your-provider.example/v1
LLM_NAME=your-apertus-model-id
```

From the repository root, run:

```
make -C track_2a run WEB_PORT=8082
```

Open `http://localhost:8082`. Docker builds the Python 3.12 application image; secrets, generated results and model weights are excluded. Fixed-feedback demo mode is available through `make -C track_2a web-demo` and makes no AI calls. Detailed deployment instructions are in `track_2a/README.md`.

Voice readiness. Language-tagged output segments and hooks for confirmed transcription input provide a boundary for future speech services. Audio capture, speech recognition, speech synthesis and model streaming are not implemented.

8. Next steps

We see improving grounding before adding more workflow complexity as the next useful step. Reactions and company replies should use only facts supported by the answer or posting, with explicit unknown handling. Follow-up questions need a stronger check that they address a real gap and stay within the current topic. Feedback should remain short, concrete and proportionate to the evidence.

A focused evaluation set with independently checked examples would help distinguish genuine improvements from changes that merely sound different. Further work should also test exercise usefulness, revision feedback, consistency across runs and local deployment requirements. Voice support can build on the existing interface once the text interaction is dependable.

As a personal note, I'd like to mention that this project has given me practical experience building, evaluating and packaging an LLM-based application, while showing the limits of relying on inference and structural checks alone. I would be happy to continue working on it, particularly on grounded feedback and more attentive conversation. I can be reached at alisa.mihalna@gmail.com.

License

Repository code retains Apache-2.0 as declared in its LICENSE. This report does not change that licence. Organiser datasets retain their own attribution and challenge-use conditions; they are not relicensed by the application.